

## DATA STRUCTURES LABORATORY

### Program: Singly Linked List for Student Records

---

### Problem Statement

**Question:**

Write a program to create and manage a singly linked list containing student records. Each record should include the student's name, roll number, and marks. Implement the following operations:

1. **Insert** a new student record into the linked list.
2. **Delete** a student record using the roll number and name.
3. **Search** for and display a particular student's details.
4. **Display** all student records.

### Source Code (linked-list.c)

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct student {
5      char name[50];
6      int roll_no;
7      int marks;
8      struct student *next;
9  };
10
11 typedef struct student list;
12 list *head = NULL;
13
14 void insert() {
15     list *temp = (list *)malloc(sizeof(list));
16     if (temp == NULL) {
17         printf("Memory allocation failed!\n");
18         return;
19     }
20
21     printf("Enter name: ");
22     scanf("%s", temp->name);
23     printf("Enter roll no: ");
24     scanf("%d", &temp->roll_no);
25     printf("Enter marks: ");
26     scanf("%d", &temp->marks);
27
28     temp->next = head;
29     head = temp;
30
31     printf("Inserted successfully!\n\n");
32 }
33
34 void display() {
35     if (head == NULL) {
```

```
36     printf("List is empty.\n\n");
37     return;
38 }
39
40 list *curr = head;
41 printf("\n--- Records ---\n");
42 while (curr != NULL) {
43     printf("Roll No: %d | Name: %s | Marks: %d\n", curr->roll_no, curr->name,
44           curr->marks);
45     curr = curr->next;
46 }
47 printf("-----\n\n");
48
49 void search() {
50     if (head == NULL) {
51         printf("List is empty.\n\n");
52         return;
53     }
54
55     int target_roll;
56     printf("Enter roll number to search: ");
57     scanf("%d", &target_roll);
58
59     list *curr = head;
60     while (curr != NULL) {
61         if (curr->roll_no == target_roll) {
62             printf("[+] Found: Roll No: %d | Name: %s | Marks: %d\n\n", curr->
63                   roll_no, curr->name, curr->marks);
64             return;
65         }
66         curr = curr->next;
67     }
68     printf("[-] Roll No %d not found.\n\n", target_roll);
69 }
70
71 void delete_student() {
72     if (head == NULL) {
73         printf("List is empty.\n\n");
74         return;
75     }
76
77     int target_roll;
78     printf("Enter roll number to delete: ");
79     scanf("%d", &target_roll);
80
81     list *curr = head;
82     list *prev = NULL;
83
84     if (curr != NULL && curr->roll_no == target_roll) {
85         head = curr->next;
86         free(curr);
87         printf("[+] Deleted successfully!\n\n");
88         return;
89     }
90
91     while (curr != NULL && curr->roll_no != target_roll) {
92         prev = curr;
93         curr = curr->next;
94     }
95
96     if (curr == NULL) {
```

```
96     printf("[-] Roll No %d not found.\n\n", target_roll);
97     return;
98 }
99
100 prev->next = curr->next;
101 free(curr);
102 printf("[+] Deleted successfully!\n\n");
103 }
104
105 int main() {
106     int choice;
107
108     while (1) {
109         printf("1. Insert\n2. Display\n3. Search\n4. Delete\n5. Exit\n");
110         printf("Choice: ");
111         scanf("%d", &choice);
112
113         switch (choice) {
114             case 1:
115                 insert();
116                 break;
117             case 2:
118                 display();
119                 break;
120             case 3:
121                 search();
122                 break;
123             case 4:
124                 delete_student();
125                 break;
126             case 5: {
127                 list *curr = head;
128                 while (curr != NULL) {
129                     list *next_node = curr->next;
130                     free(curr);
131                     curr = next_node;
132                 }
133                 return 0;
134             }
135             default:
136                 printf("Invalid choice.\n\n");
137         }
138     }
139
140     return 0;
141 }
```

## Sample Output

```
1. Insert
2. Display
3. Search
4. Delete
5. Exit
Choice: 1
Enter name: John
Enter roll no: 101
Enter marks: 85
Inserted successfully!

1. Insert
```

```
2. Display
3. Search
4. Delete
5. Exit
Choice: 1
Enter name: Alice
Enter roll no: 102
Enter marks: 92
Inserted successfully!

1. Insert
2. Display
3. Search
4. Delete
5. Exit
Choice: 2

--- Records ---
Roll No: 102 | Name: Alice | Marks: 92
Roll No: 101 | Name: John | Marks: 85
-----

1. Insert
2. Display
3. Search
4. Delete
5. Exit
Choice: 3
Enter roll number to search: 101
[+] Found: Roll No: 101 | Name: John | Marks: 85

1. Insert
2. Display
3. Search
4. Delete
5. Exit
Choice: 3
Enter roll number to search: 105
[-] Roll No 105 not found.

1. Insert
2. Display
3. Search
4. Delete
5. Exit
Choice: 4
Enter roll number to delete: 101
[+] Deleted successfully!

1. Insert
2. Display
3. Search
4. Delete
5. Exit
Choice: 2

--- Records ---
Roll No: 102 | Name: Alice | Marks: 92
-----

1. Insert
2. Display
3. Search
4. Delete
5. Exit
Choice: 5
```